

Satay Movement

Canopy

HALBORN

Satay Movement - Canopy

Prepared by:  HALBORN

Last Updated 01/28/2025

Date of Engagement by: November 22nd, 2024 - December 5th, 2024

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
14	0	0	4	3	7

TABLE OF CONTENTS

- 1. Introduction
- 2. Assessment summary
- 3. Test approach and methodology
- 4. Risk methodology
- 5. Scope
- 6. Assessment summary & findings overview
- 7. Findings & Tech Details
 - 7.1 Front-running in router assignment could cause a dos

- 7.2 Inconsistent coin-to-share conversion logic in market withdrawals
- 7.3 Vector misalignment in fund deployment
- 7.4 Miscalculation of losses
- 7.5 Inconsistent locked profit updates
- 7.6 Inconsistent fee calculation between base asset and shares
- 7.7 Misaligned management fee calculation
- 7.8 Unauthorized manipulation of profits and loss in harvest functions
- 7.9 Potential inconsistencies in profit/loss calculation due to sweep function
- 7.10 Overflow risk in deposit limit validation logic
- 7.11 Lack of metadata validation for rewards in harvest function
- 7.12 Lack of maximum lock duration validation
- 7.13 Lack of harvest functionality for fungible assets
- 7.14 Ambiguity in strategy shares metadata

1. Introduction

Canopy engaged **Halborn** to conduct a security assessment of some packages for **Movement** blockchain, beginning on November 22nd, 2024, and ending on December 13th, 2024. This security assessment focused on the smart contracts within the **Satay-movement GitHub** repository, commit hashes, and further details can be found in the Scope section of this report.

Satay Finance is a yield aggregator on the Movement network that enables efficient interaction with multiple DeFi protocols through modular strategies composed of predefined operations. It also provides automated vaults that optimize fund allocations and rebalance strategy debt ratios, offering a streamlined approach to yield generation with reduced user intervention.

2. Assessment Summary

The team at Halborn assigned two full-time security engineers to verify the security of the smart contracts. The security engineer is a blockchain and smart-contract security expert with advanced penetration testing, smart-contract hacking, and deep knowledge of multiple blockchain protocols.

The purpose of this assessment is to:

- Ensure that smart contract functions operate as intended.
- Identify potential security issues with the smart contracts.

In summary, Halborn identified some improvements to reduce the likelihood and impact of risks, which were completely addressed by the **Canopy team**. The main ones are the following:

- Implement input validation for `approve_pending_router` to prevent malicious overwrites of router address.
- Update market withdrawal functions to use `coins_to_shares` for accurate calculations based on total market supply rather than personal account values.
- Adjust `get_capacities` to include zero-capacity strategies to ensure correct pairing between strategies and their capacities during fund allocation.
- Exclude `debt_offset` from `total_loss` calculation to reflect accurate user losses and avoid unnecessary assertion failures during withdrawals.

3. Test Approach And Methodology

Halborn performed a combination of the manual view of the code and automated security testing to balance efficiency, timeliness, practicality, and accuracy regarding the scope of the smart contract assessment. While manual testing is recommended to uncover flaws in logic, process, and implementation, automated testing techniques help enhance the coverage of smart contracts. They can quickly identify items that do not follow security best practices. The following phases and associated tools were used throughout the term of the assessment:

- Research into the architecture, purpose, and use of the platform.
- Manual code review and walkthrough.
- Manual assessment of the critical Move variables and functions in scope to identify any vulnerability classes related to arithmetic or logic.
- Cross-contract call controls.
- Logical controls related to the platform architecture.
- Integration testing using the Aptos Framework.
- Comprehensive review of testnet deployment and transactions via the Movement blockchain explorer.

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1

Impact Metric (M_I)	Metric Value	Numerical Value
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

Severity Coefficient (C)	Coefficient Value	Numerical Value
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25

SEVERITY COEFFICIENT (<i>C</i>)	COEFFICIENT VALUE	NUMERICAL VALUE
Scope (<i>s</i>)	Changed (S:C) Unchanged (S:U)	1.25 1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9

SEVERITY	SCORE VALUE RANGE
Low	2 - 4.4
Informational	0 - 1.9

5. SCOPE

FILES AND REPOSITORY

(a) Repository: [satay-movement](#)

(b) Assessed Commit ID: [59858d1](#)

(c) Items in scope:

- [packages/blocks/echelon/sources/echelon_block.move](#)
- [packages/blocks/moveposition/sources/moveposition_block.move](#)
- [packages/interfaces/echelon/sources/lending.move](#)
- [packages/interfaces/echelon/sources/farming.move](#)
- [packages/interfaces/echelon/sources/scripts.move](#)
- [packages/router/sources/withdraw.move](#)
- [packages/router/sources/auth.move](#)
- [packages/router/sources/router.move](#)
- [packages/router/sources/deposit.move](#)
- [packages/satay/sources/vault.move](#)
- [packages/satay/sources/asset.move](#)
- [packages/satay/sources/base_strategy.move](#)
- [packages/satay/sources/constants.move](#)
- [packages/satay/sources/protocol.move](#)
- [packages/satay/sources/satay.move](#)
- [packages/strategies/echelon_simple/sources/strategy.move](#)
- [packages/strategies/moveposition_simple/sources/strategy.move](#)
- [packages/strategies/moveposition_simple/sources/basic_ticket.move](#)

Out-of-Scope: Third party dependencies and economic attacks.

REMEDIATION COMMIT ID:
8d72e33 042bc3c 7e84735 22e18cf 0682bfc 9830de7 50cd8e4 - edaf014 - f184f67 130d718 - eeb5ac6 - ce9373d 780db58 - a84c2db
Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
0	0	4	3	7

SECURITY ANALYSIS	RISK LEVEL	REMEDATION DATE
FRONT-RUNNING IN ROUTER ASSIGNMENT COULD CAUSE A DOS	MEDIUM	SOLVED - 01/07/2025
INCONSISTENT COIN-TO-SHARE CONVERSION LOGIC IN MARKET WITHDRAWALS	MEDIUM	SOLVED - 01/07/2025
VECTOR MISALIGNMENT IN FUND DEPLOYMENT	MEDIUM	SOLVED - 01/07/2025
MISCALCULATION OF LOSSES	MEDIUM	SOLVED - 01/07/2025
INCONSISTENT LOCKED PROFIT UPDATES	LOW	SOLVED - 01/07/2025
INCONSISTENT FEE CALCULATION BETWEEN BASE ASSET AND SHARES	LOW	SOLVED - 01/07/2025
MISALIGNED MANAGEMENT FEE CALCULATION	LOW	SOLVED - 01/07/2025

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
UNAUTHORIZED MANIPULATION OF PROFITS AND LOSS IN HARVEST FUNCTIONS	INFORMATIONAL	SOLVED - 01/22/2025
POTENTIAL INCONSISTENCIES IN PROFIT/LOSS CALCULATION DUE TO SWEEP FUNCTION	INFORMATIONAL	SOLVED - 01/07/2025
OVERFLOW RISK IN DEPOSIT LIMIT VALIDATION LOGIC	INFORMATIONAL	SOLVED - 01/07/2025
LACK OF METADATA VALIDATION FOR REWARDS IN HARVEST FUNCTION	INFORMATIONAL	SOLVED - 01/07/2025
LACK OF MAXIMUM LOCK DURATION VALIDATION	INFORMATIONAL	SOLVED - 01/07/2025
LACK OF HARVEST FUNCTIONALITY FOR FUNGIBLE ASSETS	INFORMATIONAL	SOLVED - 01/07/2025
AMBIGUITY IN STRATEGY SHARES METADATA	INFORMATIONAL	SOLVED - 01/07/2025

7. FINDINGS & TECH DETAILS

7.1 FRONT-RUNNING IN ROUTER ASSIGNMENT COULD CAUSE A DOS

// MEDIUM

Description

The `request_router_ref` function from the Protocol package allows any package to set its own address as the `pending_router` address. To confirm this address as the router, the governance account must execute the `approve_pending_router` function. The issue lies in two areas:

1. The inability to modify the router once it is set by `approve_pending_router`.
2. The lack of input validation in `approve_pending_router` to ensure that the stored address matches the expected one.

This creates an exploitable scenario:

1. An attacker can front-run the legitimate governance call to `approve_pending_router` by overwriting the `pending_router` value using `request_router_ref` with a malicious address before the approval occurs.
2. Once the malicious address is approved, it becomes irreversible because the operation `option::fill(&mut protocol_mut.router, pending_router)` will revert if `protocol_mut.router` is already set.

This vulnerability can lead to a **Denial of Service** (DoS) attack since any legitimate calls to core router functions such as `deposit_coin`, `deposit_fa`, `withdraw_coin`, or `withdraw_fa` will fail.

These functions internally call `get_signer_for_router(vault, router_ref)`, which verifies that the router address from the current contract matches the one stored in the Protocol package via `is_valid_router_ref(ref)`. If the router address has been maliciously set, this validation will fail, causing the system to revert.

Code Location

Code of `request_router_ref` function from **Protocol** module:

```

165 | public fun request_router_ref(router_signer: &signer): RouterRef acquires Protocol {
166 |     let router = signer::address_of(router_signer);
167 |
168 |     let protocol_mut = borrow_global_mut<Protocol>(get_address());
169 |     option::swap_or_fill(&mut protocol_mut.pending_router, router);
170 |     RouterRef { router: router }
171 | }

```

Code of `approve_pending_router` function from `Protocol` module:

```

150 | public entry fun approve_pending_router(account: &signer) acquires Protocol {
151 |     assert!(is_governance(signer::address_of(account)), ENOT_GOVERNANCE);
152 |
153 |     let protocol_mut = borrow_global_mut<Protocol>(get_address());
154 |     assert!(option::is_some(&protocol_mut.pending_router), ENO_PENDING_ROUTER);
155 |
156 |     let pending_router = option::extract(&mut protocol_mut.pending_router);
157 |     option::fill(&mut protocol_mut.router, pending_router);
158 | }

```

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:C/I:N/D:N/Y:N (6.7)

Recommendation

It is recommended to:

1. Introduce an address input parameter into the `approve_pending_router` function to verify that the `pending_router` address matches the expected one before approval.
2. Allow the router address to be overwritten (optional).

Remediation

SOLVED: The **Canopy team** has resolved this issue by implementing the recommended solutions.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/commit/8d72e338c3bae384f8d5db85ed57c57b4d7703e7>

7.2 INCONSISTENT COIN-TO-SHARE CONVERSION LOGIC IN MARKET WITHDRAWALS

// MEDIUM

Description

The `market_withdraw_coin_internal` and `market_withdraw_fa_internal` functions from `Echelon Strategy` module calculates the number of shares to withdraw based on the personal account values (`account_coins` and `account_shares`) instead of the total supply values managed by the market.

This approach introduces inconsistencies in the withdrawal logic, as the personal values might not accurately reflect the market's overall state. Consequently, this could result in incorrect withdrawal amounts, either overestimating or underestimating the value of shares withdrawn relative to the total market supply.

The module already provides a more accurate conversion mechanism, `coins_to_shares`, within the `echelon_block` module, which considers the market's total supply, but this function is not being utilized.

Code Location

Code of `market_withdraw_coin_internal` function from `Echelon Strategy` module:

```
377 fun market_withdraw_coin_internal<CoinType>(strategy_ref: &EchelonStrategy, amount: u64): Coin<CoinType> {
378     let market = strategy_ref.market;
379     let auth_ref = &strategy_ref.auth_ref;
380     let strategy_address = object::object_address(&base_strategy::auth_ref_strategy(&strategy_ref.auth_ref)
381
382     let account_coins = echelon_block::account_coins(strategy_address, market);
383     let account_shares = echelon_block::account_shares(strategy_address, market);
384
385     let shares_to_withdraw = math64::mul_div(amount, account_shares, account_coins);
386
387
```

```
    echelon_block::withdraw<CoinType>(&base_strategy::get_signer(auth_ref), market, shares_to_withdraw)  
}
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:M/D:M/Y:N (6.3)

Recommendation

It is recommended to replace the current calculation logic with the `coins_to_shares` function provided by the `echelon_block` module. This ensures that the coin-to-share conversion is consistent across the market and aligns with the total issued supply.

Remediation

SOLVED: The **Canopy team** has resolved this issue by implementing the recommended solutions.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/commit/042bc3cfc644b6a5280e722b62ec57f7b71fe49f>

7.3 VECTOR MISALIGNMENT IN FUND DEPLOYMENT

// MEDIUM

Description

The deployment functions `deploy_coin_funds` and `deploy_fa_funds` rely on the `get_capacities` function to allocate funds across strategies based on their available capacity.

However, the `get_capacities` function excludes strategies with zero capacity, causing the capacities vector to be shorter than the strategies vector. This misalignment leads to incorrect pairing of strategies and capacities during fund allocation.

For instance, if `strategies` contains [Strategy A, Strategy B, Strategy C] and `capacities` contains [Capacity for A, Capacity for C], the function `deploy_coin_funds` will incorrectly allocate Capacity for C to Strategy B, potentially exceeding limits or underfunding strategies.

Code Location

Code snippet of `get_capacities` function from `Deposit` module (Router package):

```
116 | vector::for_each(  
117 |     vault::vault_strategies(vault),  
118 |     |strategy| {  
119 |         let strategy_debt = vault::strategy_debt(vault, strategy);  
120 |         let strategy_debt_limit = vault::strategy_debt_limit(vault, strategy);  
121 |  
122 |         if (strategy_debt_limit > strategy_debt) {  
123 |             let capacity = strategy_debt_limit - strategy_debt;  
124 |  
125 |             vector::push_back(&mut capacities, capacity);  
126 |             total_capacity = total_capacity + capacity;  
127 |         }  
128 |     }
```

```
129 |     }  
    | );
```

Code snippet of `deploy_coin_funds` function from `Deposit` module (Router package):

```
84 | public(friend) fun deploy_coin_funds<CoinType>(  
85 |     vault: Object<Vault>,  
86 |     packets: SimpleMap<Object<BaseStrategy>, vector<u8>>  
87 | ) {  
88 |     let total_idle = vault::total_idle(vault);  
89 |     let strategies = vault::vault_strategies(vault);  
90 |     if (total_idle == 0 || vector::is_empty(&strategies)) return;  
91 |  
92 |     let (total_capacity, capacities) = get_capacities(vault);  
93 |     if (vector::length(&capacities) > 0) {  
94 |         for (i in 0..vector::length(&capacities)) {  
95 |             let capacity = *vector::borrow(&capacities, i);  
96 |             let strategy = *vector::borrow(&strategies, i);
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:M/D:M/Y:N (6.3)

Recommendation

It is recommended to:

- Include strategies with zero capacity in the `capacities` vector to maintain alignment with the `strategies` vector.
- Add validation checks in `deploy_coin_funds` and `deploy_fa_funds` to ensure that `strategies` and `capacities` vectors have the same length before allocation.
- Introduce a control in the deployment loop (`for` loop) to skip strategies with zero capacity to avoid unnecessary fund allocation operations.

Remediation

SOLVED: The **Canopy team** has resolved this issue by implementing the recommended solutions.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/commit/7e847352cda138c4b1f15f516313bec51f7a5e54>

7.4 MISCALCULATION OF LOSSES

// MEDIUM

Description

The `complete_withdrawal` function in the `Vault` module calculates `total_loss` by including `debt_offset_with_loss`, which itself includes `total_debt_offset`.

However, `debt_offset` represents funds that are successfully withdrawn by the user from the market and should not be considered a loss from the user perspective. While the protocol sees this as a reduction in vault `total_debt` and the strategy `current_debt`, treating it as a user loss misrepresents the actual outcome of the withdrawal process.

This miscalculation leads to inflated `total_loss` values, which can trigger the `max_loss` assertion unnecessarily. This means legitimate withdrawal requests may fail even when there is no real excessive loss to the user. Such errors could confuse users, undermine trust in the system, and disrupt protocol operations by rejecting valid transactions.

Code Location

Code snippet of `complete_withdrawal` function from `Vault` module (Satay package):

```
616 // if we still have some to withdraw in `remaining`,
617 // we need to subtract it from the total debt and regard it as a loss
618 let debt_offset_with_loss = total_debt_offset + remaining;
619
620 assert!(vault_mut.total_debt >= debt_offset_with_loss, EVAULT_INSUFFICIENT_DEBT);
621 vault_mut.total_debt = vault_mut.total_debt - debt_offset_with_loss;
622
623 let total_loss = debt_offset_with_loss + total_strategies_loss;
624 if (option::is_some(&max_loss)) {
625     let max_loss = option::destroy_some(max_loss);
626     let max_loss_value = math64::mul_div(to_withdraw, max_loss, constants::bps_max());
627 }
```

```
628 |         assert!(total_loss <= max_loss_value, ET00_MUCH_LOSS)
      |     };
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:M/I:L/D:N/Y:N (5.6)

Recommendation

It is recommended to revise the `total_loss` calculation to exclude `debt_offset_with_loss` and instead define it as the sum of `total_strategies_loss` and any `remaining` amount.

Remediation

SOLVED: The **Canopy team** has resolved this issue by implementing the recommended solutions.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/commit/22e18cf9340cb21f539353cdf900c60215c49ecb>

7.5 INCONSISTENT LOCKED PROFIT UPDATES

// LOW

Description

In both **Vault** and **BaseStrategy** modules, inconsistencies in updating the **total_locked** field compromise the accuracy and reliability of locked profit mechanisms:

1. Vault Module:

- In the **report** function, updates to the **total_locked** field are conditionally executed only when **total_fee > 0**. If no fees are applicable, the locked profit mechanism is skipped entirely, leading to inaccurate state maintenance. This oversight results in profits intended to be locked for future use remaining unaccounted for, causing discrepancies in the vault's state.

2. BaseStrategy Module:

- In the **complete_harvest** function, updates to the **total_locked** field are dependent on conditional blocks (**if loss > 0** or **if profit > 0**). Specifically:
 - When **loss > 0**, the function bypasses the invocation of **get_locked_profit**, which calculates and releases locked funds based on elapsed time.
 - When neither **loss > 0** nor **profit > 0**, the **total_locked** field is not updated, resulting in stale or incorrect values.

These inconsistencies in both modules disrupt the intended functionality of locked profits, potentially leading to inaccurate profit/loss reporting, an outdated locked profit state, and deviations from expected business logic.

Code Location

Code snippet of **report** function from **Vault** module (Satay package):

```
687 | if (total_fee > 0) {
688 |     let vault_ref = borrow_vault(vault);
689 |     let fee_shares = convert_amount_to_shares(vault_ref, total_fee);
690 |     let fee_shares_asset = asset::mint(vault_ref.shares_metadata, fee_shares);
691 | }
```

```

691
692 let protocol_fee_bps = protocol::get_protocol_fee();
693 if (protocol_fee_bps > 0) {
694     let protocol_fee = math64::mul_div(total_fee, protocol_fee_bps, constants::bps_max());
695     if (protocol_fee > 0) {
696         // Give the protocol or governance it's own share of the profit
697         let protocol_fee_shares = fungible_asset::extract(&mut fee_shares_asset, protocol_fee);
698         fungible_asset::deposit(
699             protocol::get_protocol_fee_recipient_store(vault_ref.shares_metadata),
700             protocol_fee_shares
701         );
702     };
703 };
704
705 // fee_shares_asset now contains the remaining fees, so we \
706 // give the remaining fee to the strategy manager/strategist
707 primary_store_deposit(base_strategy::manager(strategy), fee_shares_asset);
708
709
710 let vault_mut = borrow_vault_mut(vault);
711 let locked_profit = get_locked_profit(vault_mut) + (profit - total_fee);
712 if (locked_profit > loss) {
713     vault_mut.total_locked = locked_profit - loss;
714 } else {
715     vault_mut.total_locked = 0;
716 };
717 };

```

Code snippet of `complete_harvest` function from **Base Strategy** module (Satay package):

```

339 if (loss > 0) {
340     assert!(strategy_mut.total_debt >= loss, EINSUFFICIENT_DEBT);
341     strategy_mut.total_debt = strategy_mut.total_debt - loss;
342

```

```

343     if (strategy_mut.total_locked > loss) {
344         strategy_mut.total_locked = strategy_mut.total_locked - loss;
345     } else {
346         strategy_mut.total_locked = 0;
347     };
348 } else if (profit > 0) {
349     let (protocol_fee, manager_fee) = get_fee_shares(strategy_mut, profit);
350     let total_fee = protocol_fee + manager_fee;
351
352     assert!(total_fee <= profit, EINSUFFICIENT_DEBT);
353
354     if (protocol_fee > 0) {
355         let recipient = protocol::get_protocol_fee_recipient();
356         let protocol_fee_asset = asset::safe_mint(
357             &strategy_signer,
358             strategy_mut.shares_metadata,
359             protocol_fee
360         );
361         primary_fungible_store::deposit(recipient, protocol_fee_asset);
362     };
363
364     if (manager_fee > 0) {
365         let manager_fee_asset = asset::safe_mint(
366             &strategy_signer,
367             strategy_mut.shares_metadata,
368             manager_fee
369         );
370         primary_fungible_store::deposit(strategy_mut.manager, manager_fee_asset);
371     };
372
373     strategy_mut.total_debt = strategy_mut.total_debt + profit;
374     strategy_mut.total_locked = get_locked_profit(strategy_mut) + (profit - (total_fee));
375 };

```

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:M (3.8)

Recommendation

It is recommended to:

- **Vault Module:**
 - Move the `total_locked` update logic outside the `if total_fee > 0` condition in the `report` function. This ensures the locked profit mechanism is consistently applied, regardless of fee applicability.
- **BaseStrategy Module:**
 - Consolidate the `total_locked` calculation and update it at the end of the `complete_harvest` function, regardless of whether `profit > 0` or `loss > 0`. Use the `get_locked_profit` function consistently to ensure proper handling of time-based locked fund releases.

Remediation

SOLVED: The **Canopy team** has resolved this issue by implementing the recommended solutions.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/commit/0682bfc5a968c61d5712c1a2cab016ac76ef5aa6>

7.6 INCONSISTENT FEE CALCULATION BETWEEN BASE ASSET AND SHARES

// LOW

Description

Across the **Vault** and **BaseStrategy** modules, there are inconsistencies in fee calculations, mixing base assets and shares in the computations, which can lead to potential inaccuracies in protocol operations:

1. **Vault Module** (**report** function): The protocol fee is incorrectly calculated using the **total_fee** expressed in the base asset instead of the **fee_shares**, which represents the actual quantity of shares. Since the fee extraction is performed on **fee_shares_asset**, this mismatch may result in the protocol receiving an inaccurate number of shares, either more or fewer than intended.
2. **BaseStrategy Module** (**complete_harvest** function): The profit (in base asset) is reduced by **total_fee** (in shares), introducing an inconsistency. As a result, this operation mixes values denominated in different units, which can lead to incorrect updates of **strategy_mut.total_locked**. If **total_fee** is larger than **profit**, this may result in an underflow.

Additionally, there is a logical flaw in the assertion **assert!(total_fee <= profit, EINSUFFICIENT_DEBT)**. This condition is problematic because **total_fee** is expressed in shares, while **profit** is in the base asset. Since fees are calculated as a percentage of profit, it is not valid to directly compare these values.

Code Location

Code snippet of **report** function from **Vault** module (**Satay** package):

```
687 | if (total_fee > 0) {
688 |     let vault_ref = borrow_vault(vault);
689 |     let fee_shares = convert_amount_to_shares(vault_ref, total_fee);
690 |     let fee_shares_asset = asset::mint(vault_ref.shares_metadata, fee_shares);
691 |
692 |     let protocol_fee_bps = protocol::get_protocol_fee();
693 |     if (protocol_fee_bps > 0) {
694 |         let protocol_fee = math64::mul_div(total_fee, protocol_fee_bps, constants::bps_max());
```

```

695         if (protocol_fee > 0) {
696             // Give the protocol or governance it's own share of the profit
697             let protocol_fee_shares = fungible_asset::extract(&mut fee_shares_asset, protocol_fee);
698             fungible_asset::deposit(
699                 protocol::get_protocol_fee_recipient_store(vault_ref.shares_metadata),
700                 protocol_fee_shares
701             );
702         };
703     };

```

Code snippet of `complete_harvest` function from `Base Strategy` module (Satay package):

```

347 } else if (profit > 0) {
348     let (protocol_fee, manager_fee) = get_fee_shares(strategy_mut, profit);
349     let total_fee = protocol_fee + manager_fee;
350
351     assert!(total_fee <= profit, EINSUFFICIENT_DEBT);
352
353     if (protocol_fee > 0) {
354         let recipient = protocol::get_protocol_fee_recipient();
355         let protocol_fee_asset = asset::safe_mint(
356             &strategy_signer,
357             strategy_mut.shares_metadata,
358             protocol_fee
359         );
360         primary_fungible_store::deposit(recipient, protocol_fee_asset);
361     };
362
363     if (manager_fee > 0) {
364         let manager_fee_asset = asset::safe_mint(
365             &strategy_signer,
366             strategy_mut.shares_metadata,
367

```



```

368         manager_fee
369     );
370     primary_fungible_store::deposit(strategy_mut.manager, manager_fee_asset);
371 };
372
373     strategy_mut.total_debt = strategy_mut.total_debt + profit;
374     strategy_mut.total_locked = get_locked_profit(strategy_mut) + (profit - (total_fee));
375 };
376
377 strategy_mut.last_harvest = timestamp::now_seconds();
    }

```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (2.5)

Recommendation

It is recommended to standardize fee calculations to ensure unit consistency:

- In the **Vault** module, calculate protocol fees directly in terms of **fee_shares** rather than the base asset.
- In the **BaseStrategy** module, ensure that all calculations involving **profit** and **total_fee** are performed in the same unit.

Remediation

SOLVED: The **Canopy team** has resolved this issue by implementing the recommended solutions. The fix has been implemented in two commits:

- 9830de7
- 6711303

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/commit/9830de7480636e4b0df1c9fb656285b5c3096940>

7.7 MISALIGNED MANAGEMENT FEE CALCULATION

// LOW

Description

The current implementation of the **management fee** logic done in the `assess_fee` function from **Vault** module, ties fee calculation to the presence of profit.

This approach deviates from the intended business requirements, which specify that the management fee should be calculated based on time and assets under management (AUM), regardless of whether a profit exists.

As a result, the system fails to account for scenarios where management fees should be applied despite a lack of profit, potentially leading to under-collection of fees and a mismatch with expected product behavior.

Code Location

Code snippet of `assess_fee` function from **Vault** module (**Satay** package):

```
1294     if (duration > 0 && management_fee_bps > 0) {
1295         let current_debt = base_strategy::get_current_debt(state);
1296
1297         let management_fee =
1298             (((current_debt as u128) * (management_fee_bps as u128) * (duration as u128))
1299              / ((constants::bps_max() as u128) * (constants::seconds_per_year() as u128)) as u64);
1300
1301         total_fee = total_fee + management_fee;
1302     };
1303
1304     // Take performance fee only if there is gain and performance fee is not zero.
1305     let performance_fee_bps =
1306         option::destroy_with_default(vault_ref.performance_fee, protocol::get_performance_fee());
1307
```

```
1308     if (profit > 0 && performance_fee_bps > 0) {
1309         let performance_fee = (profit * performance_fee_bps) / constants::bps_max();
1310         total_fee = total_fee + performance_fee;
1311     };
1312
1313     math64::min(total_fee, profit)
1314 }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (2.5)

Recommendation

It is recommended to revise the management fee logic to align with the intended business requirements. Specifically, the fee calculation should depend on both the duration and the assets under management, independent of the profit value.

Remediation

SOLVED: The **Canopy team** has resolved this issue by separating the **management_fee** calculation from the **performance_fee** in the **assess_fee** function, ensuring the **management_fee** is kept independent of the profit.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/commit/50cd8e4e07570a2b0ad7330d6b0e0a30bc2e1ecf>

7.8 UNAUTHORIZED MANIPULATION OF PROFITS AND LOSS IN HARVEST FUNCTIONS

// INFORMATIONAL

Description

The `report_harvest_profit` and `report_harvest_loss` functions from `Base Strategy` module lack proper access control mechanisms.

The `HarvestRequest` object is an ephemeral struct that must be used before the transaction ends and it can only be generated by the strategy manager. However, this does not prevent a potentially malicious strategy manager from crafting a script to call these functions and inflate the `profit` or `loss` of the strategy. Such manipulation could lead to incorrect strategy states, unfair rewards, or undeserved penalties.

Code Location

Code of `report_harvest_profit` and `report_harvest_loss` functions from `Base Strategy` module (Satay package):

```
323 | public fun report_harvest_profit(request: &mut HarvestRequest, amount: u64) {  
324 |     request.profit = request.profit + amount;  
325 | }  
326 |  
327 | public fun report_harvest_loss(request: &mut HarvestRequest, amount: u64, ) acquires BaseStrategy {  
328 |     let strategy_ref = borrow_strategy(&request.strategy);  
329 |     request.loss = request.loss + amount;  
330 |     assert!(strategy_ref.total_debt >= request.loss, EINSUFFICIENT_DEBT);  
331 | }
```

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:H/D:H/Y:N (1.9)

Recommendation

It is recommended to enforce strict access control in the `report_harvest_profit` and `report_harvest_loss` functions. This can be achieved by requiring validation of the `AuthRef` that ensures that the call comes from the Strategy module, not from a script.

Remediation

SOLVED: The **Canopy team** has solved this issue by adding validation for the `AuthRef` object in each function.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/commit/edaf0143033a4d4760174135d8e735a28919ab8d>

7.9 POTENTIAL INCONSISTENCIES IN PROFIT/LOSS CALCULATION DUE TO SWEEP FUNCTION

// INFORMATIONAL

Description

The `sweep` function from the `Vault` module allows authorized accounts to transfer assets, including strategy shares, from the vault; however, transferring base assets is explicitly not permitted.

However, strategy shares are integral to calculating the profit and loss values for the vault, as used in the `get_strategy_returns` and `report` functions. Sweeping these shares can artificially alter the reported profits or losses, leading to inaccuracies in financial reporting or potentially enabling malicious exploitation by `authorized` users.

This issue becomes critical because the profit and loss calculations are essential for determining fee distributions, user share values, and the overall integrity of the vault. Altered profit/loss data may result in inflated or reduced fees, incorrect locked profit values, or discrepancies in user withdrawals.

Code Location

Code of `get_strategy_returns` function from `Vault` module (Satay package):

```
1259 fun get_strategy_returns(vault: Object<Vault>, strategy: Object<BaseStrategy>): (u64, u64) acquires Vault {
1260     let vault_ref = borrow_vault(vault);
1261     assert!(simple_map::contains_key(&vault_ref.strategies, &strategy), EVAULT_STRATEGY_MISMATCH);
1262
1263     // Get the vault's strategy shares balance...
1264     // and convert the balance to the strategy's base asset.
1265     let strategy_shares = get_strategy_shares_balance(vault, strategy);
1266     if (strategy_shares == 0) return (0, 0);
1267
1268     let strategy_assets = base_strategy::shares_to_amount(strategy, strategy_shares);
1269     let state = strategy_state(vault, &strategy);
```

```
1270     let strategy_debt = base_strategy::get_current_debt(state);
1271
1272     let (profit, loss) = (0, 0);
1273     if (strategy_assets > strategy_debt) {
1274         profit = strategy_assets - strategy_debt;
1275     } else {
1276         loss = strategy_debt - strategy_assets;
1277     };
1278
1279     (profit, loss)
1280 }
```

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:H (1.5)

Recommendation

It is recommended to restrict the **sweep** function from transferring strategy shares. Ensure it only allows assets unrelated to profit/loss calculations to be swept.

Remediation

SOLVED: The **Canopy team** has resolved this issue by implementing the recommended solutions.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/commit/f184f6781a85b727fc9ddb12f8bc68228d95a44c>

7.10 OVERFLOW RISK IN DEPOSIT LIMIT VALIDATION LOGIC

// INFORMATIONAL

Description

The `max_deposit` function from `Vault` module, used in the `deposit` function, does not account for the current deposits in the vault when no `deposit_limit` is set. This omission allows the `deposit` function to accept deposits that may exceed the vault's intended capacity. Specifically:

1. When `deposit_limit` is `None`, `max_deposit` returns `u64::max_value()`, disregarding the current asset total. This creates a potential overflow scenario.
2. The `deposit` function relies on `max_deposit` for validation, meaning the absence of proper checks in `max_deposit` directly compromises the deposit validation logic.

This oversight could lead to vault instability or inconsistent states, particularly in cases of large or repetitive deposits.

Code Location

Code of `max_deposit` function from `Vault` module (`Satay` package):

```
1023 public fun max_deposit(vault: &Vault): u64 {
1024     let deposit_limit = vault.deposit_limit;
1025     if (option::is_some(&deposit_limit)) {
1026         let limit = option::destroy_some(deposit_limit);
1027         let total_assets = get_total_assets(vault);
1028         if (limit > total_assets) {
1029             return limit - total_assets
1030         };
1031
1032         return 0
1033     };
1034 }
```



```
1035 |         return constants::u64_max()
1036 |     }
```

BVSS

AO:A/AC:L/AX:H/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (0.8)

Recommendation

It is recommended to update the `max_deposit` function to always consider the current total assets in the vault, even when `deposit_limit` is not set.

Remediation

SOLVED: The **Canopy team** has resolved this issue by implementing the recommended solutions.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/commit/130d718fda6b40e6e08730ea7e94b84b590dddcdd>

7.11 LACK OF METADATA VALIDATION FOR REWARDS IN HARVEST FUNCTION

// INFORMATIONAL

Description

The **harvest** function from **Echelon** strategy module does not validate that the metadata of the rewards matches the base asset metadata before reinvesting them into the market.

This omission creates a potential inconsistency, as invalid or incorrect rewards might be reinvested. This could result in the strategy maintaining inaccurate accounting, leading to mismanagement of the vault's funds, and exposing the system to operational or financial risks.

Code Location

Code of **harvest** function from **Echelon Strategy** module:

```
294 public entry fun harvest<CoinType>(account: &signer, strategy: Object<BaseStrategy>) acquires EchelonStrate
295     let strategy_ref = borrow_strategy(strategy);
296     let request = base_strategy::request_harvest(account, &strategy_ref.auth_ref);
297
298     let strategy_address = object::object_address(&strategy);
299     let market_address = object::object_address(&strategy_ref.market);
300     let rewards = echelon_block::get_supply_reward<CoinType>(strategy_address, market_address);
301     if (rewards > 0) {
302         let strategy_signer = base_strategy::get_signer(&strategy_ref.auth_ref);
303         let reward = echelon_block::claim_supply_reward<CoinType>(&strategy_signer, market_address);
304         base_strategy::report_harvest_profit(&mut request, coin::value(&reward));
305         market_deposit_coin_internal(strategy_ref, reward);
306     };
307
308     base_strategy::complete_harvest(request);
309 }
```

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (0.5)

Recommendation

It is recommended to implement a validation check in the **harvest** function to ensure that the metadata of the rewards matches the base asset metadata of the strategy. If the metadata does not match, the function should revert the transaction to prevent reinvestment of invalid rewards.

Remediation

SOLVED: The **Canopy team** has resolved this issue by implementing the recommended solutions.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/commit/eeb5ac6b42d1dc86844002419c9821ea94b4fde8>

7.12 LACK OF MAXIMUM LOCK DURATION VALIDATION

// INFORMATIONAL

Description

The `set_lock_duration` function, present in both the `Vault` and `Base Strategy` modules, allows authorized governance accounts to set the `lock_duration` for a strategy or vault.

While the function validates that the `duration` is above the minimum lock duration, it does not enforce an upper limit, even though a maximum lock duration exists in the `constants.move` file.

This oversight in both modules can lead to excessively large lock durations, disrupting user access to their profits.

Code Location

Code of `set_lock_duration` function from `Vault` module (Satay package):

```
441 public(friend) fun set_lock_duration(account: &signer, vault: Object<Vault>, duration: u64) acquires Vault
442     let vault_mut = borrow_vault_mut(vault);
443     assert!(protocol::is_governance(signer::address_of(account)), ENOT_AUTHORIZED);
444
445     assert!(duration >= constants::min_lock_duration(), EINVALID_LOCK_DURATION);
446
447     vault_mut.lock_duration = duration;
448     emit(SetLockDuration { vault, duration });
449 }
```

BVSS

AO:S/AC:L/AX:H/R:N/S:U/C:N/A:N/I:N/D:N/Y:H (0.5)

Recommendation

To mitigate this issue, both the Vault and Base Strategy modules should enforce a validation check for the maximum lock duration.

Remediation

SOLVED: The **Canopy team** has resolved this issue by implementing the recommended solutions.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/commit/ce9373d3489be25ae3caf9b3b74bd90e8590e0db>

7.13 LACK OF HARVEST FUNCTIONALITY FOR FUNGIBLE ASSETS

// INFORMATIONAL

Description

The **harvest** function only supports operations for **CoinType** and does not include functionality for fungible assets (FA).

This limitation restricts the applicability of the strategy to asset types that fall under **CoinType**, excluding scenarios where fungible assets are used. This could lead to operational inefficiencies and reduce the versatility of the strategy, especially in ecosystems where fungible assets play a significant role.

Score

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to implement another **harvest** function which includes support for fungible assets (FA).

Remediation

SOLVED: The **Canopy team** has resolved this issue by implementing the recommended solutions.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/commit/780db582d3e247ab9a878ab4b919d3393e9b3502>

7.14 AMBIGUITY IN STRATEGY SHARES METADATA

// INFORMATIONAL

Description

The `create_shares_asset` function in the `BaseStrategy` module uses the same `symbol` as the base asset and a `name` that resembles vault shares metadata.

While this does not cause any functional conflict in the code, as metadata is managed through distinct objects, it could lead to user confusion in scenarios where this metadata is displayed in a user interface (UI).

For instance:

- **Symbol Confusion:** The strategy shares use the same symbol as the base asset, which could mislead users into thinking they are directly dealing with the base asset.
- **Name Ambiguity:** The name of the strategy shares closely resembles that of vault shares, potentially causing users to misidentify their holdings or the source of their shares.

This confusion could result in errors when users interact with the system, particularly in scenarios involving deposits, withdrawals, or transfers.

Code Location

Code of `create_shares_asset` function from `Base Strategy` module (Satay package):

```
745 fun create_shares_asset(strategy_signer: &signer, base_metadata: Object<Metadata>): Object<Metadata> {  
746     let symbol = fungible_asset::symbol(base_metadata);  
747     let decimals = fungible_asset::decimals(base_metadata);  
748     let name = utf8(b"Satay ");  
749     string::append(&mut name, fungible_asset::name(base_metadata));
```

Code of `create_shares_asset` function from `Vault` module (`Satay` package):

```
1240 fun create_shares_asset(vault_signer: &signer, base_metadata: Object<Metadata>): Object<Metadata> {  
1241     let name = string::utf8(b"Satay ");  
1242     string::append(&mut name, fungible_asset::name(base_metadata));  
1243  
1244     let symbol = string::utf8(b"s");  
1245     string::append(&mut symbol, fungible_asset::symbol(base_metadata));  
1246  
1247     let decimals = fungible_asset::decimals(base_metadata);
```

Score

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

To mitigate potential user confusion, it is recommended to:

1. **Distinct Symbols:** Use a unique prefix or suffix for the strategy shares symbol to differentiate them from the base asset (e.g., "ss[AssetSymbol]").
2. **Clear Naming:** Include explicit identifiers in the strategy shares name, such as "Strategy Shares [Base Asset Name].".

Remediation

SOLVED: The **Canopy team** has resolved this issue by implementing the recommended solutions.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/commit/a84c2dbc019afd087ceddd7502492b5a95319631>

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.

